中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计与算法　　**批改人：**

实验	3 Pthreads 并行矩阵乘法与数组求和	专业（方向）	计算机科学与技术
学号	23336058	姓名	丁子睿
Email	dingzr@mail2.sysu.edu.cn	完成日期	2026.4.17

1. 实验目的

本实验旨在掌握 Pthreads 多线程编程模型的核心 API 与共享内存并行程序设计方法，实现基于 Pthreads 的并行矩阵乘法与并行数组求和算法;通过测试不同线程数量、数据规模下的程序性能，分析数据划分方式、结果聚合机制对并行效率的影响；探究共享内存并行程序的加速比、并行效率与可扩展性，理解线程同步开销、缓存局部性、硬件核心数限制对并行性能的制约，提升多线程程序的设计、调试与性能分析能力。

2. 实验过程和核心代码

本实验基于 Pthreads 线程库实现两类并行算法：并行矩阵乘法包含行块划分、列块划分、k 维归约三种数据 / 任务划分方式；并行数组求和包含原子操作累加、互斥锁局部求和、树型归约三种结果聚合方式，所有实验均采用固定随机种子，通过截尾平均消除单次运行的随机误差。

2.1 并行矩阵乘法

方法一：数据切分-行块划分（row-block）

并行策略：将结果矩阵 C 按行维度均匀划分成若干连续行块，每个线程分配一个行块并独立负责对应行的计算。线程可完整读取矩阵 A 的目标行、矩阵 B 的全部列数据，在本地完成乘加运算后直接写入结果矩阵 C 的对应行区域；由于各线程操作的 C 矩阵行区域互不重叠，无需线程间同步操作，避免了写冲突。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    int m;
    int n;
    int k;
    const std::vector<float>* a;
    const std::vector<float>* b;
    std::vector<float>* c;
};

// 行分块矩阵乘法核心线程函数
void* worker_row_block(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    // 计算当前线程负责的行范围
```

```
    const int row_begin = static_cast<int>((1LL * arg->tid * arg->m) /
arg->num_threads);
    const int row_end = static_cast<int>((1LL * (arg->tid + 1) * arg->m) /
arg->num_threads);

    // 按行分块计算矩阵乘法
    for (int i = row_begin; i < row_end; ++i) {
        float* c_row = &(*(arg->c))[static_cast<size_t>(i) * arg->k];
        for (int p = 0; p < arg->n; ++p) {
            const float a_ip = (*(arg->a))[static_cast<size_t>(i) * arg->n + p];
            const float* b_row = &(*(arg->b))[static_cast<size_t>(p) * arg->k];
            for (int j = 0; j < arg->k; ++j) {
                c_row[j] += a_ip * b_row[j];
            }
        }
    }
    return nullptr;
}

// 单次运行核心逻辑
double run_once(int m, int n, int k, int threads, const std::vector<float>& a, const
std::vector<float>& b,
               std::vector<float>& c) {
    std::fill(c.begin(), c.end(), 0.0f);
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
    std::vector<ThreadArg> args(static_cast<size_t>(threads));

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行行分块矩阵乘法
    for (int t = 0; t < threads; ++t) {
        args[static_cast<size_t>(t)] = {t, threads, m, n, k, &a, &b, &c};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr, worker_row_block,
&args[static_cast<size_t>(t)]);
    }
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }
    auto t1 = std::chrono::steady_clock::now();

    return std::chrono::duration<double>(t1 - t0).count();
}
```

方法二：列块划分（col_block）

并行策略：将结果矩阵 C 按列维度均匀划分成若干连续列块，每个线程分配一个列块并独立负责对应列的计算。线程可完整读取矩阵 A 的全部行数据、矩阵 B 的目标列数据，在本地完成乘加运算后直接写入结果矩阵 C 的对应列区域；由于各线程操作的 C 矩阵列区域互不重叠，无需线程间同步操作，避免了写冲突。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    int m;
    int n;
    int k;
    const std::vector<float>* a;
    const std::vector<float>* b;
    std::vector<float>* c;
};

// 列分块矩阵乘法核心线程函数
void* worker_col_block(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    // 计算当前线程负责的列范围
    const int col_begin = static_cast<int>((1LL * arg->tid * arg->k) /
arg->num_threads);
    const int col_end = static_cast<int>((1LL * (arg->tid + 1) * arg->k) /
arg->num_threads);

    // 按列分块计算矩阵乘法
    for (int i = 0; i < arg->m; ++i) {
        float* c_row = &(*(arg->c))[static_cast<size_t>(i) * arg->k];
        for (int p = 0; p < arg->n; ++p) {
            const float a_ip = (*(arg->a))[static_cast<size_t>(i) * arg->n + p];
            const float* b_row = &(*(arg->b))[static_cast<size_t>(p) * arg->k];
            for (int j = col_begin; j < col_end; ++j) {
                c_row[j] += a_ip * b_row[j];
            }
        }
    }
    return nullptr;
}

// 单次运行核心逻辑
double run_once(int m, int n, int k, int threads, const std::vector<float>& a, const
std::vector<float>& b,
               std::vector<float>& c) {
    std::fill(c.begin(), c.end(), 0.0f);
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
    std::vector<ThreadArg> args(static_cast<size_t>(threads));

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行列分块矩阵乘法
    for (int t = 0; t < threads; ++t) {
        args[static_cast<size_t>(t)] = {t, threads, m, n, k, &a, &b, &c};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr, worker_col_block,
&args[static_cast<size_t>(t)]);
    }
```

```
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }
    auto t1 = std::chrono::steady_clock::now();

    return std::chrono::duration<double>(t1 - t0).count();
}
```

方法三：k 维归约（k_block_reduce）

并行策略：将矩阵乘法的中间乘加维度（即矩阵 A 的列维度 / 矩阵 B 的行维度，记为 n）均匀分段，每个线程分配一段维度区间并独立计算该区间对结果矩阵的局部贡献，生成一个与最终结果矩阵维度一致的局部结果矩阵。各线程计算过程无写冲突、无需同步；所有线程完成局部计算后，主线程将所有局部结果矩阵逐元素求和，得到最终的结果矩阵 C。该方法需额外内存存储各线程的局部结果矩阵。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    int m;
    int n;
    int k;
    int p_begin;
    int p_end;
    const std::vector<float>* a;
    const std::vector<float>* b;
    std::vector<float>* partial_c;
};

// K 维度分块矩阵乘法核心线程函数
void* worker_k_block_reduce(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    std::vector<float>& pc = *(arg->partial_c);

    // 按 K 维度分块计算矩阵乘法
    for (int i = 0; i < arg->m; ++i) {
        float* pc_row = &pc[static_cast<size_t>(i) * arg->k];
        for (int p = arg->p_begin; p < arg->p_end; ++p) {
            const float a_ip = (*(arg->a))[static_cast<size_t>(i) * arg->n + p];
            const float* b_row = &(*(arg->b))[static_cast<size_t>(p) * arg->k];
            for (int j = 0; j < arg->k; ++j) {
                pc_row[j] += a_ip * b_row[j];
            }
        }
    }
    return nullptr;
}

// 单次运行核心逻辑
```

```
double run_once(int m, int n, int k, int threads, const std::vector<float>& a, const
std::vector<float>& b,
               std::vector<float>& c) {
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
    std::vector<ThreadArg> args(static_cast<size_t>(threads));
    std::vector<std::vector<float>> partials(static_cast<size_t>(threads),
std::vector<float>(static_cast<size_t>(m) * k, 0.0f));

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行分块矩阵乘法
    for (int t = 0; t < threads; ++t) {
        const int p_begin = static_cast<int>((1LL * t * n) / threads);
        const int p_end = static_cast<int>((1LL * (t + 1) * n) / threads);
        args[static_cast<size_t>(t)] = {t, threads, m, n, k, p_begin, p_end, &a, &b,
&partials[static_cast<size_t>(t)]};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr,
worker_k_block_reduce, &args[static_cast<size_t>(t)]);
    }
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }

    // 归约所有局部结果到最终矩阵 C
    std::fill(c.begin(), c.end(), 0.0f);
    for (int t = 0; t < threads; ++t) {
        const std::vector<float>& pc = partials[static_cast<size_t>(t)];
        for (size_t idx = 0; idx < c.size(); ++idx) {
            c[idx] += pc[idx];
        }
    }
    auto t1 = std::chrono::steady_clock::now();

    return std::chrono::duration<double>(t1 - t0).count();
}
```

2.1 并行矩阵乘法

方法一：原子操作累加（atomic_element）

并行策略：将数组按元素数量均匀划分成若干连续块，每个线程分配一个数组块并遍历其中元素。线程遍历过程中,通过原子加法操作直接将每个元素的值累加到全局求和变量中。该方法实现简单，但原子操作本质上是串行化执行的，大量原子操作会引发严重的性能瓶颈，线程数越多性能损耗越明显。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    const std::vector<int>* a;
    std::atomic<long long>* global_sum;
```

```
};

// 原子操作逐元素求和核心线程函数
void* worker_sum_atomic_element(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    const size_t n = arg->a->size();
    const size_t begin = (static_cast<size_t>(arg->tid) * n) / arg->num_threads;
    const size_t end = (static_cast<size_t>(arg->tid + 1) * n) / arg->num_threads;

    // 逐元素原子累加（无局部和，直接更新全局）
    for (size_t i = begin; i < end; ++i) {
        arg->global_sum->fetch_add((*(arg->a))[i], std::memory_order_relaxed);
    }
    return nullptr;
}

// 单次运行核心逻辑
double run_once(int threads, const std::vector<int>& a, long long& sum_out) {
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
    std::vector<ThreadArg> args(static_cast<size_t>(threads));
    std::atomic<long long> global_sum(0);

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行求和
    for (int t = 0; t < threads; ++t) {
        args[static_cast<size_t>(t)] = {t, threads, &a, &global_sum};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr,
worker_sum_atomic_element, &args[static_cast<size_t>(t)]);
    }
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }
    auto t1 = std::chrono::steady_clock::now();

    sum_out = global_sum.load(std::memory_order_relaxed);
    return std::chrono::duration<double>(t1 - t0).count();
}
```

方法二：互斥锁局部求和（mutex_local）

并行策略：将数组按元素数量均匀划分成若干连续块，每个线程分配一个数组块。线程首先遍历自身负责的数组块，在本地完成局部和的计算（无同步开销）；待局部和计算完成后，通过互斥锁保护全局求和变量的临界区，仅将局部和一次性累加到全局和中。该方法大幅减少了同步操作的次数（每个线程仅一次加锁 / 解锁），相比原子操作累加显著降低了同步开销。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    const std::vector<int>* a;
```

```
    long long* global_sum;
    pthread_mutex_t* mutex;
};

// 互斥锁+局部和求和核心线程函数
void* worker_sum_mutex_local(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    const size_t n = arg->a->size();
    const size_t begin = (static_cast<size_t>(arg->tid) * n) / arg->num_threads;
    const size_t end = (static_cast<size_t>(arg->tid + 1) * n) / arg->num_threads;

    // 1. 计算局部和（减少锁竞争）
    long long local_sum = 0;
    for (size_t i = begin; i < end; ++i) {
        local_sum += (*(arg->a))[i];
    }

    // 2. 加锁更新全局和
    pthread_mutex_lock(arg->mutex);
    *(arg->global_sum) += local_sum;
    pthread_mutex_unlock(arg->mutex);
    return nullptr;
}

// 单次运行核心逻辑
double run_once(int threads, const std::vector<int>& a, long long& sum_out) {
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
    std::vector<ThreadArg> args(static_cast<size_t>(threads));
    pthread_mutex_t mutex;
    pthread_mutex_init(&mutex, nullptr);
    long long global_sum = 0;

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行求和
    for (int t = 0; t < threads; ++t) {
        args[static_cast<size_t>(t)] = {t, threads, &a, &global_sum, &mutex};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr,
worker_sum_mutex_local, &args[static_cast<size_t>(t)]);
    }
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }
    auto t1 = std::chrono::steady_clock::now();

    pthread_mutex_destroy(&mutex);
    sum_out = global_sum;
    return std::chrono::duration<double>(t1 - t0).count();
}
```

方法三：树型归约（tree_reduce）

并行策略：将数组按元素数量均匀划分成若干连续块，每个线程分配一个数组块并先独立计算本地局部和，将局部和存入共享的局部结果数组。所有线程完成局部和计算后，通过屏障同步进入树型归约阶段：线程按固定步长两两配对，将其中一个线程的局部和合并至另一个线程的结果中；重复该归约过程，最终由 0 号线程的局部结果得到全局和。该方法同步次数最少，能最大程度发挥并行优势，并行度最高。

```
// 线程参数结构
struct ThreadArg {
    int tid;
    int num_threads;
    const std::vector<int>* a;
    std::vector<long long>* partial;
    pthread_barrier_t* barrier;
};

// 树归约求和核心线程函数
void* worker_sum_tree_reduce(void* ptr) {
    ThreadArg* arg = static_cast<ThreadArg*>(ptr);
    const size_t n = arg->a->size();
    const size_t begin = (static_cast<size_t>(arg->tid) * n) / arg->num_threads;
    const size_t end = (static_cast<size_t>(arg->tid + 1) * n) / arg->num_threads;

    // 1. 计算局部和
    long long local_sum = 0;
    for (size_t i = begin; i < end; ++i) {
        local_sum += (*(arg->a))[i];
    }
    (*(arg->partial))[static_cast<size_t>(arg->tid)] = local_sum;

    // 等待所有线程完成局部和计算
    pthread_barrier_wait(arg->barrier);

    // 2. 树归约合并局部和
    for (int stride = 1; stride < arg->num_threads; stride <<= 1) {
        if ((arg->tid % (stride << 1)) == 0) {
            const int peer = arg->tid + stride;
            if (peer < arg->num_threads) {
                (*(arg->partial))[static_cast<size_t>(arg->tid)] +=
(*(arg->partial))[static_cast<size_t>(peer)];
            }
        }
        pthread_barrier_wait(arg->barrier);
    }
    return nullptr;
}

// 单次运行核心逻辑
double run_once(int threads, const std::vector<int>& a, long long& sum_out) {
    std::vector<pthread_t> tids(static_cast<size_t>(threads));
```

```
    std::vector<ThreadArg> args(static_cast<size_t>(threads));
    std::vector<long long> partial(static_cast<size_t>(threads), 0);
    pthread_barrier_t barrier;
    pthread_barrier_init(&barrier, nullptr, static_cast<unsigned>(threads));

    auto t0 = std::chrono::steady_clock::now();
    // 创建线程执行求和
    for (int t = 0; t < threads; ++t) {
        args[static_cast<size_t>(t)] = {t, threads, &a, &partial, &barrier};
        pthread_create(&tids[static_cast<size_t>(t)], nullptr,
worker_sum_tree_reduce, &args[static_cast<size_t>(t)]);
    }
    // 等待线程结束
    for (int t = 0; t < threads; ++t) {
        pthread_join(tids[static_cast<size_t>(t)], nullptr);
    }
    auto t1 = std::chrono::steady_clock::now();

    pthread_barrier_destroy(&barrier);
    sum_out = partial[0];
    return std::chrono::duration<double>(t1 - t0).count();
}
```

3. 实验结果

3.1 测试环境与数据说明

本次实验运行于 6 核 Ubuntu 22.04 虚拟机，测试矩阵规模为 128、256、512、1024、2048，数组规模为 1M、8M、32M、64M、128M，线程数设置为 1~16。所有数据均为 5 次运行的截尾平均值，计时单位为毫秒，吞吐量单位为 GB/s，加速比以单线程运行时间为基准计算。

3.2 并行矩阵乘法性能分析

3.2.1 线程数对整体性能的影响

从所有矩阵乘法策略的整体表现来看，线程数与物理核心数的匹配度是决定性能的首要因素。当线程数从 1 增加到 4 时，所有策略的运行时间均显著下降，并行加速效果明显，这是因为线程数与虚拟机物理核心数匹配，计算负载能够被有效分配，并行收益远大于线程调度开销。当线程数超过 4 后，加速效果普遍放缓，部分小规模矩阵甚至出现性能反转，核心原因是线程数超过物理核心数引发超额订阅，导致进程频繁上下文切换、缓存争用以及宿主机调度干扰。在本次 6 核实验环境下，4 线程是通用最优并行度，几乎在所有矩阵规模下都能达到最优或接近最优的性能表现。

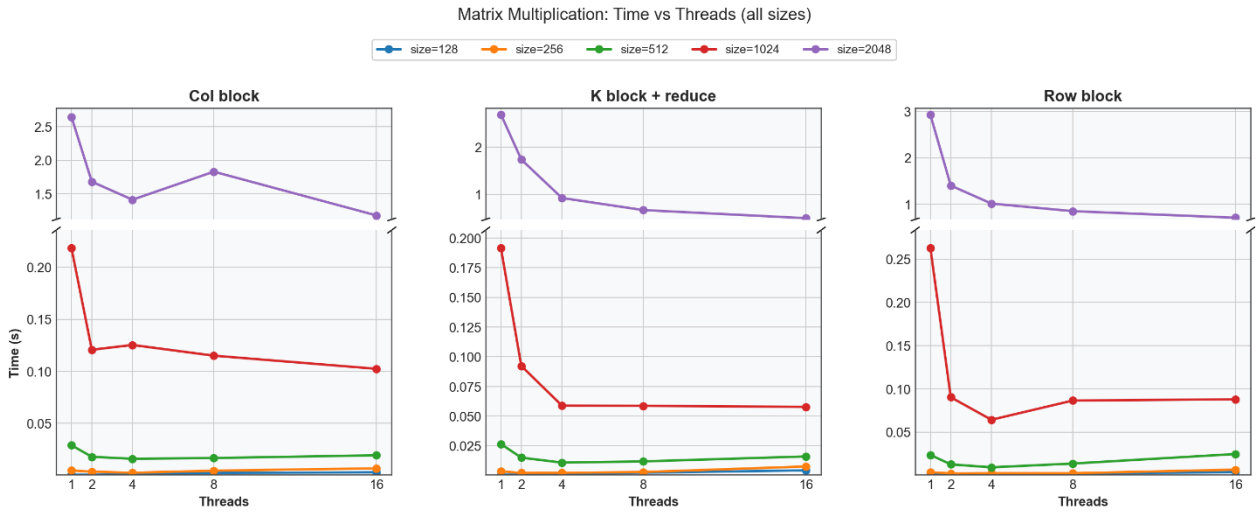

图 1 不同规模下矩阵乘法的时间随线程数变化

如图 1 的时间曲线所示，不同规模矩阵的线程扩展性存在显著差异：

小规模矩阵（128、256）：三种方法下，这两类规模的耗时都处于极低水平，且随线程数增加几乎没有下降，甚至呈现微弱上升趋势。这说明对于计算量极小的矩阵乘法，线程创建、调度带来的开销完全抵消了并行计算的收益，此时单线程运行反而是最高效的选择。

中规模矩阵（512、1024）：两类规模的耗时在线程数从 1 增加到 2~4 时出现显著下降，其中 1024 规模的降幅尤为明显，在 4 线程左右达到耗时低点；继续增加线程数至 8、16，耗时仅呈微小下降或基本保持平稳。这说明中规模矩阵的计算量足够支撑并行收益，4 线程左右即可发挥出大部分并行加速效果，后续线程数增加带来的收益边际递减。

大规模矩阵（2048）：三种方法下，该规模的耗时都随线程数增加呈现持续下降的趋势，其中 1 到 4 线程的降幅最为显著，16 线程下的耗时仅为单线程的一半左右。这说明对于计算量巨大的矩阵乘法，即使线程数超过物理核心数，并行收益依然能覆盖调度和超额订阅的开销，线程数增加带来的加速效果可以持续体现，大规模任务更能发挥多线程并行的优势。

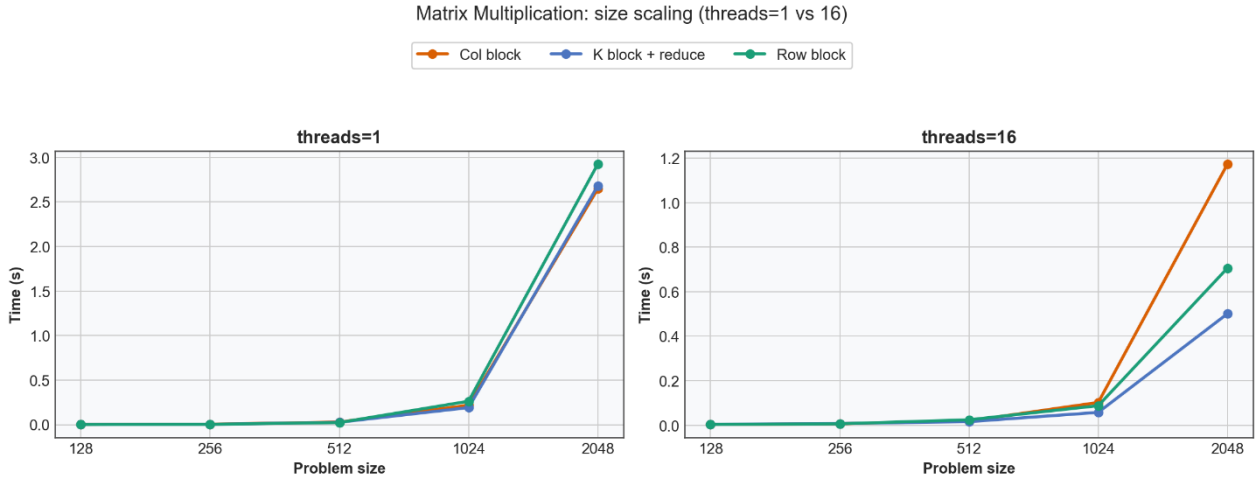

图 2 不同规模下矩阵乘法的时间随线程数变化（1 线程 vs 16 线程）

图 2 可以进一步验证问题规模对并行收益的影响：单线程下三种方法的运行时间几乎一致，且随矩阵规模呈三次方增长；16 线程下，2048 规模的并行收益最为显著，行块划分耗时仅为单线程的 24%，而 128 规模下三种方法的 16 线程耗时均高于单线程。这表明只有当计算量足够大时，并行化才有

实际意义。

3.2.2 不同划分方法的性能对比

在相同线程数配置下，三种数据划分方法的性能差异主要由缓存局部性和内存开销决定。

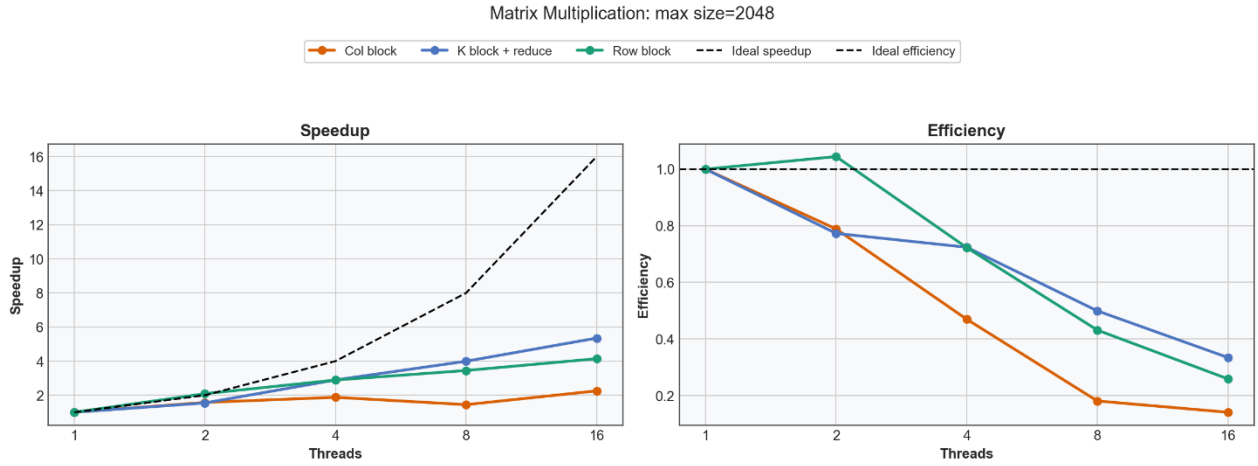

图 3 2048 规模下矩阵乘法的加速比与并行效率

为了量化三种并行策略的可扩展性，我们以 2048×2048 的大规模矩阵乘法为例，对比了不同线程数下的加速比与并行效率。加速比的计算公式为：$S(p) = T(1)/T(p)$，其中 $T(1)$ 为单线程运行时间，$T(p)$ 为 p 线程运行时间。并行效率的计算公式为：$E(p) = S(p)/p$。从图 3 中可以清晰看到三种策略的性能差异：

行块划分策略：该策略的并行效率表现最为稳健，在 2 线程下甚至出现了轻微的超线性加速，这是因为线程数增加后，数据访问更贴合 CPU 缓存行，减少了缓存缺失带来的额外开销。随着线程数增加，效率缓慢下降，16 线程下仍保持约 0.26 的效率，对应加速比约为 4.1，整体性能稳定，无明显短板；

列块划分策略：该策略的性能衰减最为明显，随着线程数增加，效率快速下降，16 线程时仅约 0.14，对应加速比仅约 2.2。这是因为列块划分需要频繁访问矩阵 B 的列数据，破坏了缓存的空间局部性，高线程数下缓存争用和伪共享问题加剧，成为严重的性能瓶颈；

k 维归约策略：该策略的加速比在高线程数下表现最优，16 线程时加速比达到约 5.4，是三种方法中最高的；对应的并行效率从 2 线程的约 0.78 缓慢下降至 16 线程的约 0.34，下降幅度小于列块划分。但该策略需要为每个线程分配独立的局部结果矩阵，2048 规模下的内存开销是其他两种方法的 16 倍，属于典型的“以空间换时间”的权衡。

综合来看，行块划分策略在缓存友好性、内存开销和实现复杂度之间取得了最佳平衡，是大规模矩阵乘法的通用最优方案；k 维归约在内存充足的场景下可以获得更高的加速比，适合对性能要求极高且内存资源充裕的场景；列块划分由于缓存效率问题，仅在特定场景下适用，不推荐作为首选方案。

3.3 并行数组求和性能分析

3.3.1 线程数对整体性能的影响

数组求和是典型的内存密集型任务，性能瓶颈并非计算量，而是线程同步与缓存一致性开销，这使

得其线程扩展性与计算密集型的矩阵乘法有显著差异。图 4 展示了三种聚合方式下，不同数组规模的运行时间随线程数的变化曲线，三种方法的性能表现差异巨大，同步机制的选择直接决定了并行收益的上限。

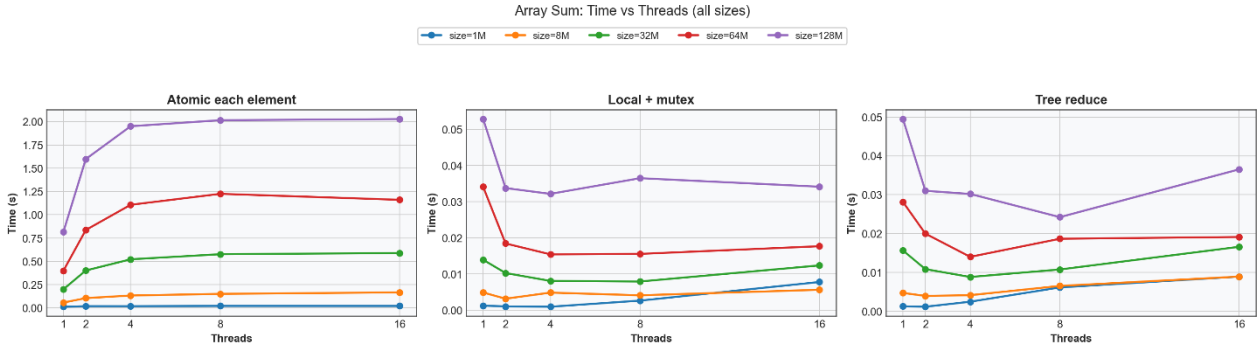

图 4 2048 不同规模下数组求和的时间随线程数变化

如图 4 所示，不同聚合方法的线程扩展性差异巨大：

原子操作累加：所有规模下，耗时都随线程数增加单调上升，且规模越大，上升幅度越剧烈：128M 规模的耗时从 1 线程的约 0.8 秒飙升至 16 线程的 2 秒以上，完全没有并行收益；即使是 1M 小规模，耗时也没有下降，反而保持平稳或微升。这是因为原子操作需要对每个元素的累加进行全局同步，严重破坏了缓存一致性，线程数越多，缓存失效和总线争用的开销越大，完全抵消了并行带来的收益；

互斥锁局部求和：曲线呈现明显的 “先降后升” 趋势：1M 小规模下，耗时随线程数增加持续上升，并行收益为负；8M 以上中大规模，耗时在线程数从 1 增加到 2~4 时显著下降，在 4 线程左右达到最低点，之后随着线程数继续增加，耗时缓慢回升。这种趋势的原因是，该方法先让每个线程计算局部和，仅在最后用一次互斥锁同步，大幅减少了同步次数；但互斥锁本身会带来串行化瓶颈，线程数过多时，锁竞争的开销会逐渐超过并行收益，导致耗时回升；

树型归约：曲线同样呈现 “先降后升” 的趋势，但最优线程数比互斥锁更高，高线程数下的扩展性也更好：1M 小规模下，耗时随线程数增加持续上升；8M 以上中大规模，耗时在 8 线程左右达到最低点，之后虽然回升，但上升幅度比互斥锁更小。这是因为树型归约采用对数级的屏障同步，将同步次数从 O (1) 降低到 O (log p)，大幅减少了同步开销，同时避免了互斥锁的串行化瓶颈，因此能在更高线程数下保持更好的性能。

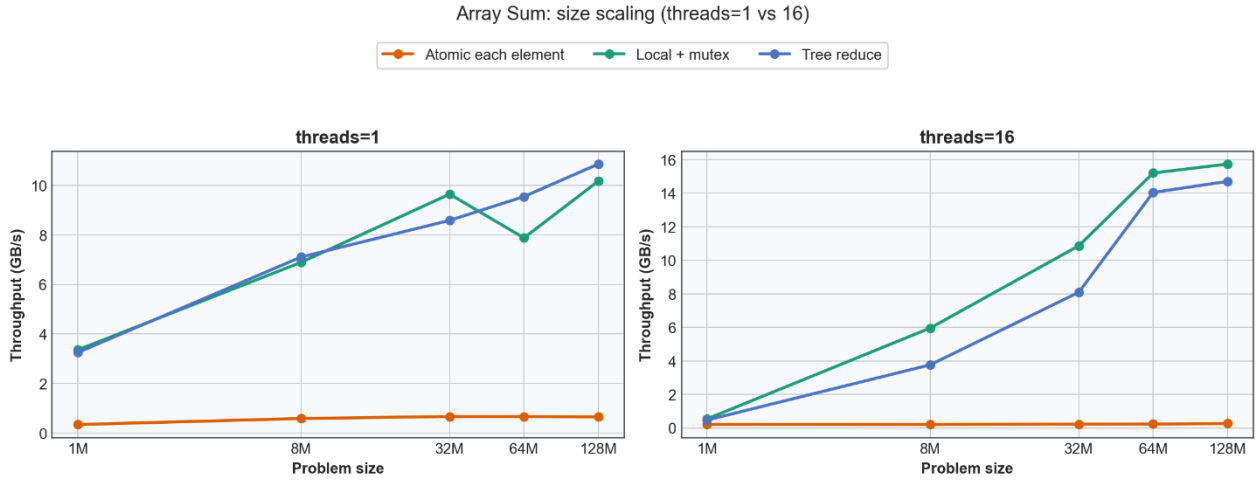

图 5 数组求和的吞吐量随问题规模变化（1 线程 vs16 线程）

图 5 对比了 1 线程与 16 线程下三种数组求和方法的吞吐量随问题规模（1M~128M）的变化趋势，直观反映了不同同步策略对内存带宽利用率与规模扩展性的影响。

单线程下，原子操作的吞吐量始终维持在极低水平，几乎不随规模变化，而互斥锁局部求和与树型归约的吞吐量几乎完全重合，随数组规模增大呈线性增长，在 128M 规模时达到约 10.85GB/s，说明单线程下两种方法均能高效利用内存带宽，原子操作则因串行化开销成为性能瓶颈。

16 线程下，三种方法的性能差异显著，原子操作的吞吐量仍维持在极低水平，无任何并行收益，互斥锁局部求和与树型归约的吞吐量则随规模增大快速上升，在 128M 规模时分别达到约 15.73GB/s 和 14.04GB/s，均显著高于单线程，其中互斥锁在大规模下的吞吐量略高，而树型归约的曲线斜率更大、扩展性更优，同时可以看到小规模数组下，16 线程的吞吐量均低于单线程，并行收益为负，而 8M 以上中大规模数组下，并行带来的带宽提升效果才开始显现。这说明只有当计算量足够大，能覆盖线程创建与同步开销时，多线程并行才能发挥内存密集型任务的性能优势。

3.3.2 不同聚合方法的性能对比

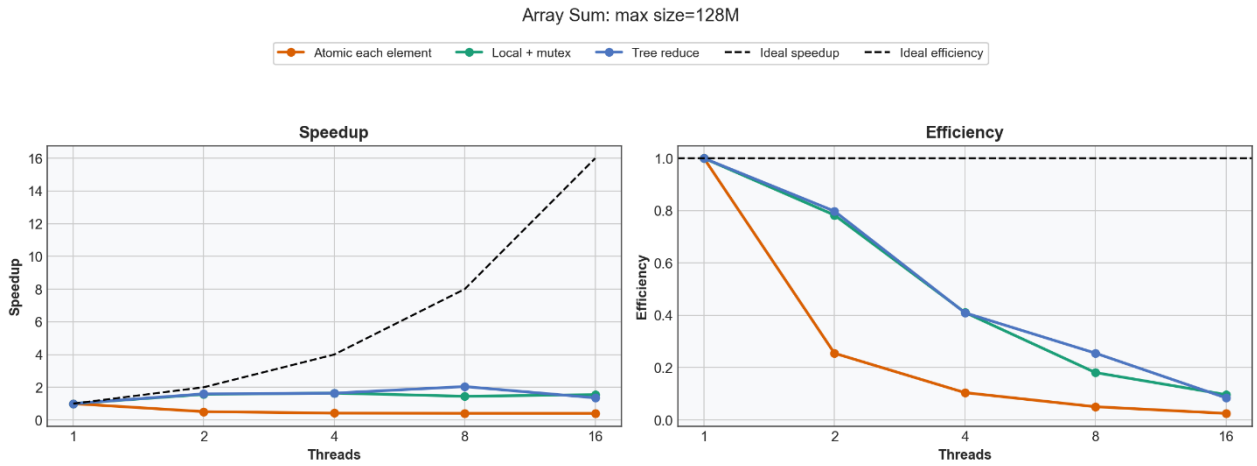

图 6 128M 规模下数组求和的加速比与并行效率

如图 6 所示，三种方法的加速比与效率计算公式与前文相同，差距极为显著：

原子操作累加：加速比始终小于 1，且随线程数增加持续下降，16 线程下加速比仅为 0.4，完全失去并行意义。这是因为每个元素的累加都需要原子操作，完全串行化了计算过程，缓存一致性协议的开销成为性能瓶颈；

互斥锁局部求和：加速比在 4 线程达到峰值 1.64，之后缓慢下降。该方法通过先计算局部和再同步的方式，将同步次数从 O (n) 减少到 O (1)，大幅降低了同步开销，但互斥锁的串行化特性使得高线程数下性能提升有限；

树型归约：加速比在 8 线程达到峰值 2.04，是所有方法中最高的，且其并行效率始终高于互斥锁，16 线程下仍保持 0.09。该方法采用对数次屏障同步和无锁归约机制，充分利用了多核并行能力，是本次实验的最优方案。

综合来看，树型归约策略在同步开销和并行度之间取得了最佳平衡，是大规模数组求和的首选方案；互斥锁局部求和实现简单，性能接近树型归约，适合快速开发场景；原子操作累加应完全避免使用。

3.4 实验结论

本次实验表明，共享内存并行程序的性能高度依赖于数据划分方式和同步机制。对于计算密集型的矩阵乘法，行块划分策略因最优的缓存局部性成为首选方案；对于内存密集型的数组求和，树型归约策略因最少的同步开销性能最优。并行线程数应不超过物理核心数，本次 6 核环境下 4 线程是性价比最高的选择。原子操作和互斥锁等同步机制会引入显著的串行化开销，应尽量减少同步次数，采用局部计算后归约的方式提升并行效率。

4. 实验感想

通过本次 Pthreads 并行编程实验，我深入理解了共享内存并行与分布式内存并行的本质差异。与 MPI 的显式通信不同，Pthreads 通过共享内存实现数据交换，但线程同步的复杂性和开销成为性能瓶颈。实验中不同策略的性能对比让我深刻认识到，缓存局部性和同步开销是决定共享内存并行程序性能的核心因素，简单地增加线程数并不能带来线性加速。

本次实验也让我掌握了多线程程序的性能分析方法，学会了通过控制变量法定位性能瓶颈，理解了硬件资源对并行性能的制约。这些经验为我后续开发高性能并行程序奠定了坚实的基础。